Lecture 1: Overview & Tokenization

Building Large Foundation Models from Scratch · WS 2026/27

Asst. Prof. Dr. Martin Pawelczyk

Faculty of Computer Science, University of Vienna

Thu 1 Oct 2026

Today

1. Why this course, and how it works
2. What is a language model?
3. Tokenization
   - From text to bytes
   - Byte-pair encoding
   - Design decisions and quirks
   - Beyond BPE
4. Wrap-up

How every lecture works

- Today: 90 minutes, no quiz.
- From Lecture 2 on: a **15-minute quiz** driven by the "questions to think about" from the previous lecture, then **70 minutes** of new material.
- Tuesday sessions are in the PC room.

Why this course, and how it works

1. Why this course, and how it works
2. What is a language model?
3. Tokenization
   - From text to bytes
   - Byte-pair encoding
   - Design decisions and quirks
   - Beyond BPE
4. Wrap-up

Why build a foundation model from scratch?

- **The operating-systems argument.** Nobody writes their own OS at work, but building one is fastest way to learn what every layer does.
- **Frontier models are closed.** Weights, data and recipes are secret; the public knows less about frontier training today than in 2020.
- **But the recipe is public enough.** Llama 3, DeepSeek, Qwen, OLMo, SmolLM publish what is needed to rebuild the pipeline at small scale.
- **Abstractions obscure.** `model.generate()` hides the decisions that determine cost and quality. We remove that layer.

The motivation for this course

You understand a system when you can build a small version of it *and* predict what changes when it gets $1{,}000\times$ bigger.

Inspired by Hashimoto & Liang (CS336 at Stanford University).

Tentative questions we want to answer in eight weeks

1. **Build** an LM end to end: tokenizer, model, optimizer, training loop, sampler. **L1–L3**
2. **Account** for compute, memory and $: FLOPs, bytes, GPU-hours and tokens. **L2**, **L4**, **L5**
3. **Scale** deliberately: run a scaling experiment, fit a scaling law, predict the next model. **L6**
4. **Serve** and **measure**: why inference is a different problem from training; what benchmarks do and do not tell you. **L7**, **L8**
5. **Feed** the model: where pretraining data comes from and what filtering does to it. **L9**
6. **Post-train** it: SFT, RLHF, RLVR, and what "alignment" means. **L10**
7. **Read a technical report critically**: which decisions matter, why, and what they cost.

What we can and cannot do here

	Frontier run	This course
Training compute	10^{25}–10^{26} FLOPs	10^{16}–10^{17} FLOPs
Parameters	10^{11}–10^{12}	10^{6}–10^{8}
Training tokens	10^{13}	10^{8}–10^{9}
Hardware	10^{4}–10^{5} GPUs	one Colab GPU
Cost	10^{7}–10^{8} EUR	0–10 EUR
Team	hundreds	2–3 students

Orders of magnitude; public estimates for 2024–26 frontier runs.

What transfers across $10^{9}\times$ in compute?

- Mechanics – fully. The code is the same; only the numbers get bigger.
- Mindset – fully. Every decision is "loss per FLOP": squeeze the most out of the hardware and the data.
- Intuitions – partially. Some architecture and data choices only show at scale. That is why scaling laws exist (**L6**).

The key question

Efficiency

Given a fixed budget of **compute** (FLOPs), **data** (tokens) and **engineering time**: *which decisions produce the best model?*

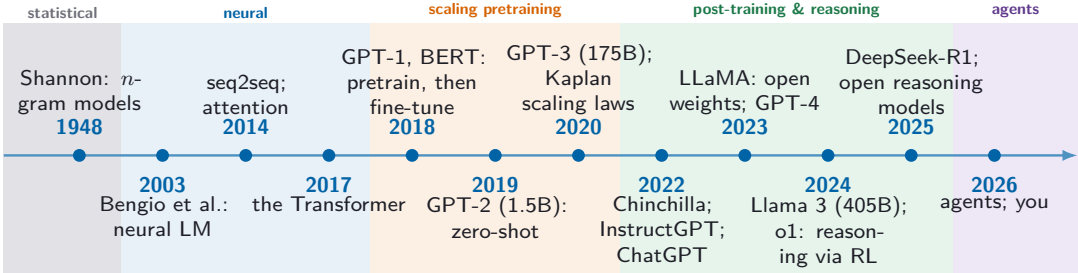
Every lecture aims to answer a piece of this:

- **L1** How many tokens does a byte of text cost?
- **L2–L3** How much loss do we buy per FLOP? (accounting, architecture)
- **L4–L5** What fraction of peak FLOP/s do we actually get, on one GPU and on a thousand?
- **L6** How should FLOPs be split between a bigger model and more data?
- **L7** What does a served token cost, and why is that so different from a trained one?
- **L8–L9** How much capability per token of data, and how do we measure it?
- **L10** How much behaviour per token of *expensive* (human-labelled, verified) data?

The "bitter lesson" is often read as "only scale matters".

Perhaps better: *algorithms that scale well matter most*, and at scale, efficiency is a key algorithm.

Seventy-five years of language modelling, in five regimes



- Regime changes: learned representations replace counts; **scale becomes a strategy**; RL becomes steers capabilities; the model becomes an actor.
- The course tries to follow a similar arc:
  pretraining the base model (**L1–L9**), then post-training it (**L10**).

“What is a language model?” has had four answers

- **2018** BERT: something you **fine-tune**. One pretrained model, one labelled dataset per task.
- **2020** GPT-3: something you **prompt**. The task described in the context window; no gradient steps.
- **2022** ChatGPT: something you **talk to**. Post-training turns a text predictor into an assistant. [example conversation ↗]
- **2026** Agents: something that **acts autonomously**. Tools, long-running tasks, verification; the model produces actions, not only text. [example trace ↗]

Takeaway

The fundamentals are the same: attention, kernels, optimisation. The specs are different: longer context, and inference efficiency matters even more.

Open models: who publishes what

Early attempts (2020–22): replicate GPT-3

- EleutherAI: The Pile (open 800 GB dataset), GPT-J 6B [Gao et al., 2020; Wang & Komatsuzaki, 2021]
- Meta: OPT 175B; famous for its logbook of hardware failures [Zhang et al., 2022]
- BigScience / Hugging Face: BLOOM 176B, multilingual, careful data sourcing [Workshop, 2022]

Open weights (weights + paper), 2023–26

- Meta: LLaMA, Llama 2, Llama 3 [Touvron et al., 2023a,b; Grattafiori et al., 2024]
- Mistral: 7B, Mixtral [Jiang et al., 2023, 2024]
- DeepSeek: LLM, V2, V3, R1, V3.2 [2024–25]
- Alibaba: Qwen2.5, Qwen3 [2024, 2025]
- Moonshot: Kimi k1.5, K2 [2025, 2026]; Z.ai: GLM-4.5, GLM-5 [2025, 2026]; MiniMax M2.5, Xiaomi MiMo-V2 [2026]

Approaching the closed models (GPT, Claude, Gemini).

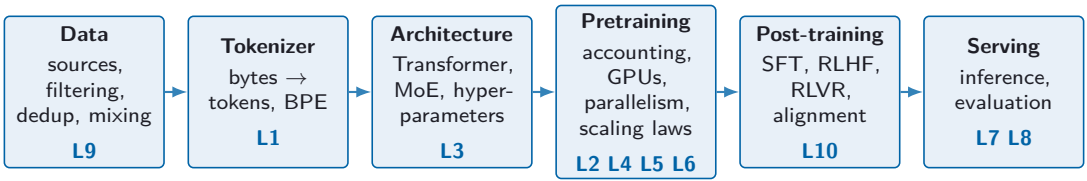
Open source (weights + paper + code + data)

- AI2: OLMo, OLMo 2, OLMo 3 [Groeneveld et al., 2024; OLMo, 2024, 2025]
- NVIDIA: Nemotron-4, Nemotron 3 [Parmar et al., 2024; NVIDIA, 2025]
- Marin (Stanford): open *development*, every run public [Marin, 2025]

Where this course's recipe comes from.

Open weights let you *run* a model, open source lets you *rebuild* one; this course follows third column.

The LM development pipeline, and where each lecture sits



- Order that allows to *start building early*: 1) tokenizer, 2) model first, 3) data later.
- The assignments cut across the pipeline:
  A1 = tokenizer → architecture → a trained model
  A2 = pretraining at several scales → a scaling law.

Schedule

#	Date	Room	Session	Assignments
1	Thu 01.10.	SR 6	L1 Overview & tokenization	
2	Tue 06.10.	PC 3	L2 Resource accounting	A1 out
3	Thu 08.10.	SR 6	L3 Architectures: dense, attention variants, MoE	
4	Tue 13.10.	PC 3	L4 GPUs	
5	Thu 15.10.	SR 6	L5 Parallelism	
6	Tue 20.10.	PC 3	L6 Scaling laws	
7	Thu 22.10.	SR 6	**Lab 1**: A1 walkthrough	A1 due, A2 out
8	Tue 27.10.	PC 3	L7 Inference	
9	Thu 29.10.	SR 6	L8 Evaluation	
10	Tue 03.11.	PC 3	L9 Data	
11	Thu 05.11.	SR 6	L10 Post-training: SFT, RLHF, RLVR, alignment	
12	Tue 10.11.	PC 3	**Lab 2**: A2 walkthrough (Colab sweeps, fitting)	A2 due
13	Thu 12.11.	SR 6	**Review session**: exam preparation	
14	Tue 17.11.	PC 3	**Mid-term exam** (L1–L10)	
15	Thu 19.11.	SR 6	Buffer or Mini-break	

SR 6 = Seminarraum 6, Währinger Straße 29 (Thu 13:15–14:45).
PC 3 = PC-Seminarraum 3, Kolingasse 14–16 (Tue 15:00–16:30).

Two assignments

A1 – A minimal LM from scratch
6.10. – 22.10.

- Byte-level BPE tokenizer: training, `encode`, `decode`
- Transformer LM (RMSNorm, RoPE, attention, SwiGLU), cross-entropy, AdamW, cosine schedule, training loop, sampling
- Train on TinyStories on one GPU; report loss curves and samples

A2 – Scaling
22.10. – 12.11.

- Your A1 model on Colab: IsoFLOP sweeps over 4–5 compute budgets (10^{14}–10^{16} FLOPs) $\times$ model sizes
- Fit a Chinchilla-style scaling law to *your own* runs
- Predict the best model at $10\times$ the compute, train it, check.

Compute: Google Colab (the free T4 is enough; Pro is faster), or any GPU you have.
Grading: mid-term exam 50 % · **TBD**: up to 5% bonus via quizzes.

How to work in this course

Collaboration

- Work in a group; discuss freely.
- Submissions are per person; everyone must be able to explain every line.

AI assistants

- Fine for conceptual and API questions.
- Not for generating your solution. Switch off AI autocomplete for the assignments! The struggle is the point.
- Do not read other people's BPE or Transformer code.

This week

- Check that a Colab GPU runtime starts for you.

Where things live

- Slides, announcements: Moodle
- Code: will be released soon
- Questions: the Moodle forum

What is a language model?

1. Why this course, and how it works
2. What is a language model?
3. Tokenization
   - From text to bytes
   - Byte-pair encoding
   - Design decisions and quirks
   - Beyond BPE
4. Wrap-up

A language model is a probability distribution over sequences

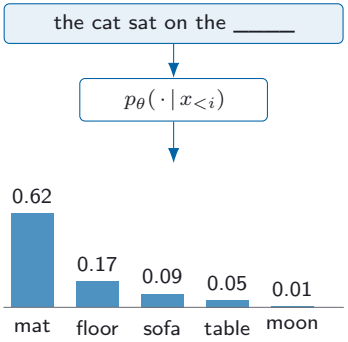
Language model

A distribution $p(x_1, \ldots, x_L)$ over token sequences; vocabulary size V.

- $p(\text{the cat sat})$ large; $p(\text{cat the sat})$ small.
- Chain rule, no approximation:

$$p(x_{1:L}) = \prod_{i=1}^{L} p(x_i \mid x_{<i})$$

- An **autoregressive** LM is a network that outputs $p(x_i \mid x_{<i})$ as a categorical distribution over V tokens.



Training objective: predict the next token

- Maximum likelihood on a corpus of N tokens = minimise the **cross-entropy** (nats per token). Every token is a labelled example: a 15T-token corpus is $1.5 \cdot 10^{13}$ of them.

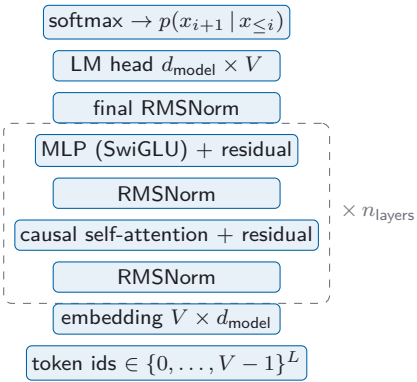
$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(x_i \mid x_{<i})$$

- **Perplexity** $= \exp(\mathcal{L})$, the effective number of equally likely next tokens. Loss $2.0 \Rightarrow$ perplexity 7.4.
- Loss depends on the tokenizer (more tokens $\Rightarrow$ lower loss *per token*). To compare tokenizers use **bits per byte**: bpb $= \frac{\mathcal{L}}{\ln 2} \cdot \frac{\#\text{tokens}}{\#\text{bytes}}$.

Takeaway

Pretraining is one giant classification problem with V classes.
The rest of the course is about doing it *efficiently*.

The Transformer architecture: an overview



- Tokens become vectors; n_{layers} identical blocks mix information **across positions** (attention) and **within a position** (MLP); a linear head maps each final vector to next-token logits.
- Causal: position i sees only positions $\le i$, so one forward pass yields all L predictions at once. That is what makes training parallel.
- Dense-block parameters $\approx 12\, n_{\text{layers}}\, d_{\text{model}}^2$ (attention $4d_{\text{model}}^2$, MLP $8d_{\text{model}}^2$).
  GPT-3: $96 \cdot 12288^2 \cdot 12 = 1.7 \cdot 10^{11}$.
- Every box is a function we try to write in A1. Details and modern variants in **L3**.

Generation: next-token prediction in a loop

```
def generate(model, ids, max_new, T=1.0, top_p=1.0):
    for _ in range(max_new):
        logits = model(ids)[-1] / T        # last pos.
        probs = nucleus(softmax(logits), top_p)
        next_id = sample(probs)
        ids.append(next_id)
        if next_id == EOT:
            break
    return ids
```

- **Temperature** T: $T \to 0$ is greedy, $T > 1$ flattens.
- **Top-k / top-p**: drop the tail; the tail is where low quality is.
- One forward pass per token, *sequentially*: inference is a different problem from training (**L7**); hence the KV cache.
- A chat model is this loop plus a prompt template (**L10**).

Scale, in numbers

Model	Params N	Tokens D	$\approx 6ND$ FLOPs	Note
Your A1 model (2026)	$2 \cdot 10^7$	$2 \cdot 10^8$	$2.4 \cdot 10^{16}$	minutes on one GPU
GPT-3 (2020)	$1.75 \cdot 10^{11}$	$3 \cdot 10^{11}$	$3.1 \cdot 10^{23}$	the "foundation model" moment
Chinchilla (2022)	$7 \cdot 10^{10}$	$1.4 \cdot 10^{12}$	$5.9 \cdot 10^{23}$	Gopher's compute, better model
Llama 3 405B (2024)	$4.05 \cdot 10^{11}$	$1.56 \cdot 10^{13}$	$3.8 \cdot 10^{25}$	$\sim$16k H100s, $\sim$2 months
DeepSeek-V3 (2024)	671B / 37B active	$1.48 \cdot 10^{13}$	$\sim 3.3 \cdot 10^{24}$	MoE: cost follows *active* N

- The $6ND$ **rule**: about 6 FLOPs per parameter per training token (2 forward, 4 backward). We will come back to it in **L2**; we will use it in many calculations from now on.
- Per training token, a frontier model spends $\sim 10^{12}$ FLOPs.
  A laptop CPU delivers $\sim 10^{11}$ FLOP/s.
- A1 to Llama 3 is nine orders of magnitude.
  Structurally, it is the same code.

Why efficiency is so important

Back of the envelope for one frontier run

- Budget: $4 \cdot 10^{25}$ FLOPs (Llama 3 405B).
- One H100: $1 \cdot 10^{15}$ FLOP/s peak in bf16, but training sustains roughly **40 % of peak** (model FLOPs utilisation, **L4**).
- Time:

$$\frac{4 \cdot 10^{25}}{0.4 \cdot 1 \cdot 10^{15}} = 1 \cdot 10^{11} \text{ GPU-s} \approx 2.8 \cdot 10^{7} \text{ GPU-h}$$

- At 2 EUR per GPU-hour: ≈ 55 M EUR of compute for one run, before any failed experiment.

Consequence

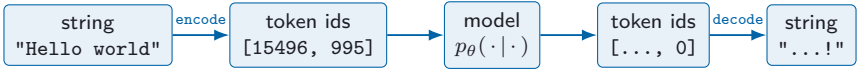
A $2\times$ efficiency gain anywhere in the stack – tokenizer, kernels, parallelism, data, scaling recipe – is worth as much as doubling the budget.

That is why this course is organised around *where the waste goes*, not around a catalogue of architectures.

Tokenization

1. Why this course, and how it works
2. What is a language model?
3. Tokenization
   - From text to bytes
   - Byte-pair encoding
   - Design decisions and quirks
   - Beyond BPE
4. Wrap-up

Where the tokenizer sits



Requirements

- Lossless: `decode(encode(s)) == s` for *any* string
- Fixed vocabulary size V: it is the size of the softmax
- Good compression: few tokens per byte of text
- Fast on terabytes; works for every language, code, emoji

```python
class Tokenizer:
    def __init__(self, vocab, merges, specials):
        ...
    def encode(self, text: str) -> list[int]: ...
    def decode(self, ids: list[int]) -> str: ...
```

The interface you implement in A1. The ids shown above are GPT-2's.

Strings are sequences of Unicode code points

- Unicode assigns an integer, the **code point**, to every character: about $155{,}000$ of them, across scripts, symbols and emoji.
- Code points run from 0 to `0x10FFFF` $= 1{,}114{,}111$; most of the range is unused.
- A Python `str` is a sequence of code points: `len("ü") == 1`.
- For a tokenizer: $155{,}000$ possible symbols, almost all extremely rare. We need something smaller and denser.

```
>>> ord("a"), ord("ü"), ord("\u4e16")
(97, 252, 19990)    # a, ü, CJK 'world'
>>> chr(127757)
'...'               # U+1F30D, Earth emoji
>>> [ord(c) for c in "Hi!"]
[72, 105, 33]
```

Code points are stored as bytes: UTF-8

Character	Code point	UTF-8 bytes (hex)	#
a	U+0061	`61`	1
ü	U+00FC	`C3 BC`	2
CJK “world”	U+4E16	`E4 B8 96`	3
Earth emoji	U+1F30D	`F0 9F 8C 8D`	4

- Variable length, 1–4 bytes; ASCII unchanged; about 98 % of the web.
- Every string maps to one byte sequence, but not every byte sequence is valid UTF-8. Decode with `errors="replace"`: a model *will* emit half an emoji.

The trick everyone uses since GPT-2

Tokenize **bytes**, not characters. The base alphabet then has exactly 256 symbols, nothing is ever out of vocabulary, and any language, emoji or binary garbage is representable.

Bit layout (the leading bits announce the length):

```
1 byte    0xxxxxxx
2 bytes   110xxxxx 10xxxxxx
3 bytes   1110xxxx 10xxxxxx 10xxxxxx
4 bytes   11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
```

Three obvious tokenizers, and why nobody uses them

	Characters	Bytes	Words
Vocabulary	$\sim$155,000 code points	256	unbounded: new words never stop appearing (Zipf)
Out of vocabulary	no	never	yes $\rightarrow$ `<UNK>`
Sequence length	1 per character, $\sim$5 per English word	$\sim 4\times$ longer than subwords; worse for non-Latin scripts	shortest
Main problem	huge, sparse embedding table	compute per byte: every position is a full forward pass, attention is $O(L^2)$	OOV; no sharing between *run* / *running* / *ran*; no spaces in many languages
Used by	early char-RNNs	research (end of lecture)	n-gram LMs, early NMT

- The word-level OOV problem is what brought BPE into NLP [Sennrich et al., 2016].

The number that matters: bytes per token

Compression ratio

$$\text{bytes per token} = \frac{\#\text{bytes (UTF-8)}}{\#\text{tokens}}$$

- Higher is better: more text per context window, more bytes per training run, fewer tokens per served character.
- Roughly **4 bytes per token** for English with a 100k–200k vocabulary; lower for most other languages, code and numbers.

Try it yourself

- `pip install tiktoken`; load `gpt2`, `cl100k_base`, `o200k_base`.
- Measure bytes per token on: an English paragraph; the same paragraph in German; a Python file; a page of Chinese Wikipedia; a list of phone numbers.
- Predict the ranking *before* you run it.

Try it out yourself.

Byte-pair encoding: learn the vocabulary from the data

- 1994: a compression algorithm [Gage]. 2016: reused in machine translation to fix OOV [Sennrich et al.].
- 2019: GPT-2 runs it on **bytes**. Byte-level BPE is still the standard: GPT-4o, Llama 3, DeepSeek, Qwen, Mistral.

The algorithm in one sentence

Start with the 256 bytes as tokens. Repeatedly merge the most frequent *adjacent* pair into a new token, until the vocabulary has V entries.

What you get

- Frequent strings (`␣the` , `ing` , `␣Vienna`) become single tokens.
- Rare strings fall back to pieces, ultimately bytes: `␣Kolingasse` → something like `␣Kol ing asse` .
- Nothing is out of vocabulary.
- The vocabulary is a *learned* compression dictionary for the training corpus, biases included.

BPE training

```
def train_bpe(corpus, vocab_size, specials):
    vocab = {i: bytes([i]) for i in range(256)}
    # ... plus one id per special token
    counts = count_pretokens(corpus)  # {str: n}
    seqs = {w: tuple(bytes([b]) for b in w.encode())
            for w in counts}
    merges = []
    while len(vocab) < vocab_size:
        pairs = Counter()
        for w, c in counts.items():
            for a, b in zip(seqs[w], seqs[w][1:]):
                pairs[(a, b)] += c
        best = max(pairs, key=lambda p: (pairs[p], p))
        merges.append(best)
        vocab[len(vocab)] = best[0] + best[1]
        seqs = {w: merge(seqs[w], best) for w in seqs}
    return vocab, merges
```

- Output: a **vocabulary** (id $\rightarrow$ bytes) and an ordered list of **merges**. *The merge list is the tokenizer.*
- This version recounts every pair after every merge: $O(\text{corpus} \times \text{merges})$.

An example

Corpus as pretoken counts: `cat` $\times 4$ `cats` $\times 3$ `car` $\times 2$ `cart` $\times 2$

Step	Pair counts (top 3)	Merge	New token
1	(c,a) 11 (a,t) 7 (a,r) 4	c+a	`ca`
2	(ca,t) 7 (ca,r) 4 (t,s) 3	ca+t	`cat`
3	(ca,r) 4 (cat,s) 3 (r,t) 2	ca+r	`car`
4	(cat,s) 3 (car,t) 2	cat+s	`cats`

After 4 merges: $V = 256 + 4$; merges $=$ [(c,a), (ca,t), (ca,r), (cat,s)].

Encoding new words with these merges

- "carts": `c a r t s` $\rightarrow$ `ca r t s` $\rightarrow$ `car t s`
- "cats": `ca t s` $\rightarrow$ `cat s` $\rightarrow$ `cats`

Takeaway

Merges are applied **in training order**; frequencies in the new text play no role.

Encoding and decoding with a trained tokenizer

```
def encode(self, text):
    ids = []
    for chunk in split_on_special_tokens(text):
        if chunk in self.special:
            ids.append(self.special[chunk]); continue
        for pre in re.findall(PAT, chunk):  # pre-tokens
            toks = [bytes([b]) for b in pre.encode()]
            for pair in self.merges:  # by rank
                toks = merge(toks, pair)
            ids.extend(self.token_to_id[t] for t in
     toks)
    return ids

def decode(self, ids):
    raw = b"".join(self.vocab[i] for i in ids)
    return raw.decode("utf-8", errors="replace")
```

- Scanning 50k merges per pretoken is wasteful: real implementations rank adjacent pairs and merge the lowest rank first (priority queue).
- Cache: `␣the` is encoded millions of times.
- `decode` is byte concatenation; invalid UTF-8 must not crash.
- `encode_iterable` streams a file so a 20 GB corpus never sits in memory.

Pre-tokenization: chop first, then merge

- Without it, BPE merges *across* words: `␣of␣the` becomes a token and the vocabulary fills with chunks that generalise badly.
- So: split into **pretokens** (words with leading space, digit runs, punctuation runs) and merge only *inside* them. GPT-2's regex:

```
PAT = (r"'s|'t|'re|'ve|'m|'ll|'d"
       r"| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+"
       r"|\s+(?!\S)|\s+")
```

`'s|'t|'re|...`	English contractions get their own token
`␣?\p{L}+`	a run of letters with an optional leading space: `␣the` $\neq$ `the`
`␣?\p{N}+`	a run of digits (GPT-2); Llama 3 and GPT-4 cap a run at 3 digits
`␣?[^\s\p{L}\p{N}]+`	punctuation and symbols
`\s+(?!\S)`	whitespace, but a final space stays with the next word

Special tokens

- Documents are concatenated for training; `<|endoftext|>` marks the boundary, so model learns context does not carry across it.
- A special token must be *one* id: split on special tokens **before** pre-tokenization and never merge across them. Chunking the corpus there also makes parallel pre-tokenization trivially correct.
- Chat models add more (**L10**): `<|im_start|>`, tool-call markers.
  GPT-2: $V = 50{,}257 = 256 + 50{,}000 + 1$.

A1 corner cases (tested)

- Overlapping specials such as `<|endoftext|><|endoftext|>`: longest match first.
- Specials are exempt from the regex.
- Empty string; a single space; invalid UTF-8 on decode.

Engineering: from ten hours to ten minutes

Training

- Pre-tokenize once; count *unique* pretokens. Zipf: 2 GB of text has $\sim 5 \cdot 10^8$ words but $\sim 10^6$ distinct pretokens.
- Parallelise pre-tokenization over chunks split at `<|endoftext|>`.
- Update pair counts *incrementally*: a merge only touches pretokens containing the pair. Keep pairs in a heap. Profile first.

Encoding

- Merge by rank with a priority queue, not by scanning the merge list.
- Cache pretoken $\rightarrow$ ids.
- Pure Python: a few MB/s; Rust with parallelism: orders of magnitude faster. At 15T tokens this is a real cost.

Takeaway

Simple algorithm; the engineering decides whether the experiment happens at all.

Design decision: vocabulary size

A bigger V buys ...

- Shorter sequences: more bytes per token, longer effective context, cheaper inference per character, better non-English coverage.
- Parameters idle for most tokens: embedding + head $= 2\,V\,d_{\text{model}}$. Llama 3 8B: $2 \cdot 128{,}256 \cdot 4096 = 1.05 \cdot 10^9$, 13 % of the model.
- Softmax cost per token $\propto V\,d_{\text{model}}$; for small models it dominates.
- Undertrained rare tokens: each token needs enough occurrences to learn its embedding.

Model	Year	V
GPT-2	2019	50,257
Llama 2	2023	32,000
GPT-4 (cl100k)	2023	∼100,000
Llama 3	2024	128,256
GPT-4o (o200k)	2024	∼200,000
DeepSeek-V3	2024	∼129,000
Gemma 2	2024	256,000

Trend: 32k → 128k–256k, driven by multilingual data and inference cost. The optimal V grows with model size [Tao et al., 2024].

Tokenizer anomalies

- **Whitespace and code.** GPT-2 encodes each space of a Python indent separately; newer tokenizers have multi-space tokens. This alone changes the cost of code by up to $2\times$.
- **Numbers.** Whether `1234567` is one token, a digit run, or 3-digit groups changes how models do arithmetic. Llama 3 and GPT-4 chose 3-digit groups deliberately.
- **Same word, different tokens.** `Hello`, `␣Hello`, `hello`, `HELLO` are unrelated ids; their relationship must be learned from data.
- **Languages are not equal.** The same content costs 2–5$\times$ more tokens in many languages than in English: costlier to train and serve, shorter effective context [Petrov et al., 2023].
- **Glitch tokens.** Strings in the vocabulary that barely occur in training (Reddit user names, debug strings): their embedding is never trained; behaviour is undefined [Rumbelow & Watkins, 2023].
- **"How many r's in strawberry?"** The model never sees letters, only `str aw berry`. Many "dumb" LM failures are tokenizer failures.

Alternatives to BPE, and why BPE are preferred

Method	Idea	Used by
Byte-level BPE	greedy merges of frequent pairs; deterministic	GPT-2/3/4, Llama 3, DeepSeek, Qwen, Mistral
WordPiece [Schuster & Nakajima, 2012]	merge the pair that most increases corpus likelihood, not the most frequent one	BERT
Unigram LM [Kudo, 2018]	start from a huge candidate set; prune tokens whose removal hurts likelihood least. Probabilistic segmentation can be *sampled* as augmentation	T5, Llama 1/2, many multilingual models
SentencePiece [Kudo & Richardson, 2018]	a library: BPE or Unigram on raw text, no regex pre-tokenizer; spaces become a visible underscore	Llama 1/2, T5, Gemma

- Quality differences are small at equal V; BPE wins on simplicity, speed and tooling. What moves the needle: byte-level base, the regex, digit handling, V, and the tokenizer's training corpus.

Do we need tokenizers at all?

- **The dream**: bytes in, bytes out. No vocabulary, no quirks, every language equal.
- **The obstacle is cost**: $4\times$ more positions means $4\times$ the per-position compute and $16\times$ the attention compute.
- **Fix: patch the bytes.** A cheap local model groups bytes into patches; the big model runs once per patch.
  - MegaByte [Yu et al., 2023]: fixed-size patches.
  - Byte Latent Transformer [Pagnoni et al., 2024]: patch boundaries where the next byte is *hard to predict*; matches Llama-3-style models at 8B.
- Publicly, no frontier model has dropped its tokenizer yet.

Honest summary

Tokenization is a lossy engineering compromise. We keep it because it works and the alternatives have not beaten it *on cost*. That is the shape of most decisions in this course.

Assignment 1, part 1: your tokenizer

You implement

- `train_bpe(path, vocab_size, special_tokens)` $\rightarrow$ `(vocab, merges)`
- `Tokenizer(vocab, merges, special_tokens)` with `encode`, `encode_iterable`, `decode`
- Train with $V = 10{,}000$ on TinyStories [Eldan & Li, 2023].

You report

- Training time, peak memory, longest token
- Bytes per token on TinyStories vs. a text of your choice; `encode` throughput in MB/s

Time allocation

Plan a couple of hours for the tokenizer, especially for making it fast enough. Start this early!

Wrap-up

1. Why this course, and how it works
2. What is a language model?
3. Tokenization
   - From text to bytes
   - Byte-pair encoding
   - Design decisions and quirks
   - Beyond BPE
4. Wrap-up

Summary

1. This course: build the whole stack yourself at small scale and reason about what changes at large scale. The organising question is **efficiency**: loss per FLOP, per token, per euro.
2. A language model is $p(x_i \mid x_{<i})$; training is cross-entropy on next tokens; the cost is $\approx 6ND$ FLOPs.
3. Text $\rightarrow$ Unicode code points $\rightarrow$ UTF-8 bytes $\rightarrow$ tokens. Byte-level BPE: 256 byte tokens plus an ordered list of merges. *The merge list is the tokenizer.*
4. Pre-tokenization (the regex) and special tokens define most of the behaviour; vocabulary size is a compute/quality trade-off, currently 128k–256k.
5. Many odd LM behaviours – arithmetic, spelling, language unfairness, glitch tokens – are tokenizer behaviours.

Next: Lecture 2 – Resource accounting

Tuesday 6 October, PC-Seminarraum 3, Kolingasse 14–16

Coming up

- Tensors, memory layout, `einops`
- Counting FLOPs: where $6ND$ comes from, and where it is wrong
- Memory: parameters, gradients, optimizer states, activations
- Precision: fp32, bf16, fp8 – what each costs and what it breaks
- Arithmetic intensity: why your GPU is usually waiting for memory

Reading

- Sennrich et al. (2016), sections 1–3
- Radford et al. (2019), section 2.2: GPT-2's byte-level BPE
- The tiktoken README and its `educational` module
- Optional: Pagnoni et al. (2024), Byte Latent Transformer

Do

- Set up the A1 repository when it's out; start tokenizer & **revise the lecture**!

References

- Hashimoto, Liang et al. Stanford CS336: Language Modeling from Scratch (Spring 2026). `https://cs336.stanford.edu/`. The course this one is modelled on.
- Shannon (1948). A mathematical theory of communication. *Bell System Technical Journal*.
- Bengio, Ducharme, Vincent & Jauvin (2003). A neural probabilistic language model. *JMLR*.
- Hugging Face (2024). SmolTalk dataset, conversation 72. `https://huggingface.co/datasets/HuggingFaceTB/smoltalk`.
- Nebius (2025). SWE-rebench OpenHands trajectories, trajectory 1. `https://huggingface.co/datasets/nebius/SWE-rebench-openhands-trajectories`.
- Gao et al. (2020). The Pile: An 800GB dataset of diverse text for language modeling. arXiv:2101.00027.
- Wang & Komatsuzaki (2021). GPT-J-6B: A 6 billion parameter autoregressive language model. EleutherAI.
- Zhang et al. (2022). OPT: Open pre-trained Transformer language models. arXiv:2205.01068.
- BigScience Workshop (2022). BLOOM: A 176B-parameter open-access multilingual language model. arXiv:2211.05100.
- Touvron et al. (2023). Llama 2: Open foundation and fine-tuned chat models. arXiv:2307.09288.
- Jiang et al. (2023). Mistral 7B. arXiv:2310.06825.
- Jiang et al. (2024). Mixtral of experts. arXiv:2401.04088.

References (cont.)

- DeepSeek-AI (2024). DeepSeek LLM: Scaling open-source language models with longtermism. arXiv:2401.02954.
- DeepSeek-AI (2024). DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model (MLA). arXiv:2405.04434.
- DeepSeek-AI (2025). DeepSeek-V3.2. arXiv:2512.02556.
- Qwen Team (2024). Qwen2.5 technical report. arXiv:2412.15115.
- Yang et al. (2025). Qwen3 technical report. arXiv:2505.09388.
- Kimi Team (2025). Kimi k1.5: Scaling reinforcement learning with LLMs. arXiv:2501.12599.
- Kimi Team (2026). Kimi K2 series technical report. arXiv:2602.02276.
- GLM-4.5 Team (2025). GLM-4.5: Agentic, reasoning, and coding foundation models. arXiv:2508.06471.
- GLM-5 Team (2026). GLM-5. arXiv:2602.15763.
- MiniMax (2026). MiniMax M2.5. `https://www.minimax.io/news/minimax-m25`.
- Xiaomi (2026). MiMo-V2 Pro. `https://mimo.xiaomi.com/mimo-v2-pro`.
- Groeneveld et al. (2024). OLMo: Accelerating the science of language models. arXiv:2402.00838.
- OLMo Team (2024). 2 OLMo 2 Furious. arXiv:2501.00656.
- Team OLMo (2025). OLMo 3. arXiv:2512.13961.

References (cont.)

- Parmar et al. (2024). Nemotron-4 15B technical report. arXiv:2402.16819.
- NVIDIA (2025). Nemotron 3. arXiv:2512.20856.
- Marin project (2025). Marin 8B and 32B retrospectives. `https://marin.readthedocs.io/`.
- Vaswani et al. (2017). Attention is all you need. *NeurIPS*.
- Radford et al. (2019). Language models are unsupervised multitask learners (GPT-2). OpenAI.
- Brown et al. (2020). Language models are few-shot learners (GPT-3). *NeurIPS*.
- Hoffmann et al. (2022). Training compute-optimal large language models (Chinchilla). arXiv:2203.15556.
- Grattafiori et al. (2024). The Llama 3 herd of models. arXiv:2407.21783.
- DeepSeek-AI (2024). DeepSeek-V3 technical report. arXiv:2412.19437.
- Gage (1994). A new algorithm for data compression. *C Users Journal*.
- Sennrich, Haddow & Birch (2016). Neural machine translation of rare words with subword units. *ACL*.
- Schuster & Nakajima (2012). Japanese and Korean voice search (WordPiece). *ICASSP*.
- Kudo (2018). Subword regularization: Improving neural network translation models with multiple subword candidates. *ACL*.

References (cont.)

- Kudo & Richardson (2018). SentencePiece: A simple and language independent subword tokenizer and detokenizer. *EMNLP* (demo).
- OpenAI. tiktoken: a fast BPE tokeniser. `https://github.com/openai/tiktoken`.
- Tao et al. (2024). Scaling laws with vocabulary: Larger models deserve larger vocabularies. *NeurIPS*.
- Petrov, La Malfa, Torr & Bibi (2023). Language model tokenizers introduce unfairness between languages. *NeurIPS*.
- Rumbelow & Watkins (2023). SolidGoldMagikarp (plus, prompt generation). LessWrong / AI Alignment Forum.
- Yu et al. (2023). MegaByte: Predicting million-byte sequences with multiscale transformers. *NeurIPS*.
- Pagnoni et al. (2024). Byte Latent Transformer: Patches scale better than tokens. arXiv:2412.09871.
- Eldan & Li (2023). TinyStories: How small can language models be and still speak coherent English? arXiv:2305.07759.